LARGE LANGUAGE MODELS (LLMs)

Complete technical notes: tokens, Transformers, training, inference, fine-tuning, and deployment

How to use these notes
Read the concepts in order, run the code snippets in a small project, and verify every claim with experiments. API names can change across library versions; consult the installed package documentation when implementing.

1. What is a Large Language Model?

A Large Language Model (LLM) is a neural network trained on large collections of text (and, in multimodal systems, other data) to model sequences of tokens. Most modern generative LLMs use the Transformer architecture. At inference time, the model predicts a distribution over the next token and generates a response one token at a time.

- **Language model:** estimates the probability of token sequences.
- **Large:** refers broadly to model scale, training data, and compute—not a single fixed parameter threshold.
- **Generative:** can produce new sequences conditioned on an input prompt.

An LLM is not a database or a guaranteed fact engine. It generates likely continuations based on learned patterns and current context; outputs must be verified for high-impact use.

2. Tokens, tokenization, and context windows

A token is a unit used by the model. It may be a whole word, part of a word, punctuation, whitespace, or a special control token. Tokenization converts text into token IDs; the model processes IDs rather than raw characters.

Why tokens matter

- Input and output limits are measured in tokens, not pages or words.
- API pricing and inference throughput are commonly token-based.
- Different languages, code, and unusual strings can tokenize differently.
- A context window is the maximum token sequence the model can consider for one request, including prompt, retrieved context, conversation history, and generated output.

```
# Conceptual tokenization flow
text = "Build a useful assistant"
token_ids = tokenizer.encode(text)
reconstructed = tokenizer.decode(token_ids)
```

Special tokens can mark message boundaries, roles, end-of-turn, tool calls, or multimodal payloads. Their exact spellings and behavior depend on the model family and chat template.

3. Neural network foundations

Parameters, weights, and layers

A neural network is a sequence of parameterized transformations. Parameters (weights and biases) are adjusted during training to reduce a loss function. A layer transforms hidden representations into new representations.

Vectors, matrices, and tensors

A scalar is one number, a vector is a one-dimensional array, a matrix is two-dimensional, and a tensor generalizes these to multiple dimensions. LLM computations are largely tensor operations.

Forward pass and loss

A forward pass maps input token IDs to logits. Logits are converted to probabilities with softmax. During training, a loss function compares the predicted next token distribution with the actual next token.

```
# Simplified next-token objective
input:  "The capital of France is"
target: " Paris"

loss = cross_entropy(model(input), target)
```

4. Transformer architecture

The Transformer uses attention and feed-forward networks to process token representations. It replaced recurrent processing in many language tasks because it supports efficient parallel training across sequence positions.

Core components

- **Token embeddings:** map token IDs to learned vectors.
- **Positional information:** supplies information about token order, using positional embeddings or positional techniques such as RoPE.
- **Self-attention:** lets each token representation incorporate information from other positions.
- **Feed-forward network (MLP):** applies nonlinear transformations to each position.
- **Residual connections and normalization:** help optimization and information flow through deep networks.
- **Output head:** maps hidden states to vocabulary logits.

Query, Key, and Value

For each token, attention computes Query (Q), Key (K), and Value (V) projections. Query-Key similarity determines how strongly a position attends to another position; the weighted Values are combined.

```
Attention(Q, K, V) =
    softmax((Q K^T) / sqrt(d_k)) V

Q = X W_Q
K = X W_K
V = X W_V
```

Causal attention masks future positions during autoregressive language modeling so a token cannot use tokens that come after it in the training sequence.

Multi-head attention

Several attention heads learn different relationships in parallel. Their outputs are combined and projected. Multi-head attention increases representational flexibility.

Encoder-only, decoder-only, encoder-decoder

- **Encoder-only:** bidirectional representations, commonly useful for classification and embeddings.
- **Decoder-only:** causal next-token generation; common for chat and code LLMs.
- **Encoder-decoder:** an encoder reads input and a decoder generates output; common in sequence-to-sequence tasks.

5. How LLMs are trained

Pretraining

Pretraining exposes a model to large datasets and optimizes a self-supervised objective. A decoder-only model often learns next-token prediction. Data quality, deduplication, filtering, mixture design, and licensing all matter.

Supervised fine-tuning (SFT)

SFT trains a pretrained model on curated prompt-response or instruction-response examples. It teaches the model to follow instructions and adopt useful response formats.

Preference optimization and alignment

Preference data compares candidate responses. Methods such as RLHF use a reward model and reinforcement learning; other approaches such as DPO optimize preference comparisons more directly. These methods aim to improve helpfulness and behavior but do not eliminate errors.

Training stages in brief

- Collect, filter, deduplicate, and tokenize data.
- Choose model architecture, objective, optimizer, and training schedule.
- Train with batches; monitor loss, throughput, and numerical stability.
- Evaluate on held-out data and task-specific benchmarks.
- Fine-tune and align for the target interaction style and safety requirements.
- Package weights, tokenizer, config, and inference runtime.

6. Inference and text generation

Inference is the process of using trained weights to generate outputs. The model produces logits for the next token; a decoding strategy selects a token; the token is appended and the process repeats until a stopping condition.

Decoding controls

- **Greedy decoding:** chooses the highest-probability token each step; deterministic but can be repetitive.
- **Temperature:** scales logits before sampling. Lower values concentrate probability; higher values increase variation.
- **Top-k:** samples only among the k most likely tokens.
- **Top-p / nucleus:** samples from the smallest set whose cumulative probability reaches p.
- **Repetition penalties:** discourage repeated tokens or phrases; excessive penalties can harm quality.
- **Stop conditions:** end-of-turn tokens, stop sequences, or maximum output length.

Temperature does not make a model more knowledgeable. It changes sampling behavior. For factual tasks, evaluate settings rather than assuming one value is always best.

7. Prompting and chat formats

A prompt provides instructions, context, examples, and the user's task. A chat model usually expects messages with roles such as system, developer, user, and assistant, serialized using a model-specific chat template.

- **System/developer instructions:** define behavior, constraints, and output expectations.
- **User message:** supplies the task and relevant inputs.
- **Few-shot examples:** demonstrate the desired input-output pattern.
- **Structured output:** request JSON or a schema and validate it programmatically.
- **Prompt injection:** malicious or conflicting instructions inside untrusted input; isolate data from instructions and limit tool permissions.

```
# Simple prompt pattern
role = "You are a concise technical tutor."
task = "Explain cosine similarity."
constraints = "Use one equation and one small example."
prompt = f"{role}\nTask: {task}\nConstraints: {constraints}"
```

8. Fine-tuning, RAG, and tools

These approaches solve different problems and can be combined.

- **RAG:** retrieves external evidence at request time; suitable for private or frequently updated facts.
- **Fine-tuning:** changes model weights; suitable for task behavior, style, and specialized patterns.
- **Tool use:** lets a model request an external action such as database lookup or calculation. The application executes and validates the tool call.
- **Long context:** places more source material directly in the prompt, but can increase cost and does not guarantee the model uses every detail.

9. Quantization and efficient inference

Quantization represents weights or activations with lower-precision numeric formats to reduce memory use and often improve inference speed. Examples include 8-bit and 4-bit weight quantization. Quality loss depends on model, method, and workload.

- **KV cache:** stores attention keys and values from prior tokens so they do not need to be recomputed at every generation step.
- **Batching:** processes multiple requests or sequences together to improve accelerator utilization.
- **Streaming:** returns generated tokens progressively to reduce perceived latency.

- **GPU memory:** must hold model weights plus runtime buffers, KV cache, and other overhead.

10. Local LLMs with Ollama

Ollama packages models and exposes a local runtime and API. It is useful for learning, prototyping, and privacy-conscious local workflows. Model availability and hardware requirements vary.

```
# Example commands
ollama list
ollama pull qwen3.5:4b
ollama run qwen3.5:4b

# Example Python using the Ollama client
from ollama import chat

response = chat(
    model="qwen3.5:4b",
    messages=[
        {"role": "user", "content": "Explain a Python decorator."}
    ],
)
print(response.message.content)
```

For application deployments, keep the model runtime behind a service boundary, configure timeouts and concurrency, and monitor memory and latency. A laptop model is not automatically suitable for production traffic.

11. Evaluating LLMs

- **Task accuracy:** correctness on a representative set of tasks.
- **Instruction following:** compliance with constraints and requested formats.
- **Groundedness:** support from provided sources when evidence is required.
- **Robustness:** behavior under paraphrases, long inputs, ambiguity, and adversarial prompts.
- **Latency and throughput:** time to first token, total response time, and tokens per second.
- **Cost:** hardware, hosting, API usage, and operational costs.

Use task-specific test sets and human review. Public benchmarks are useful signals, not a substitute for testing the real application.

12. Safety, limitations, and responsible use

- Hallucinations: validate factual claims, especially in high-impact settings.
- Bias: training data can encode harmful stereotypes or uneven coverage.
- Privacy: do not send sensitive data to a model provider without appropriate authorization and safeguards.
- Copyright and data provenance: verify rights and permitted uses of training and input data.
- Prompt injection and tool misuse: enforce permissions outside the model.
- Evaluation drift: monitor performance as prompts, models, and user behavior change.

13. Build a small LLM application

A practical first project is a technical tutor API. Build a FastAPI endpoint that accepts a question, calls a local Ollama model, streams or returns the response, validates input size, and logs latency without storing sensitive prompts unnecessarily.

```
POST /chat
{
  "message": "Explain Python generators",
  "temperature": 0.2
}

Response:
{
  "answer": "...",
  "model": "qwen3.5:4b"
```

```
}
```

14. Glossary

Parameter: A learned numeric value in a neural network.

Logit: An unnormalized score for a possible next token.

Softmax: Converts logits into a probability distribution.

Transformer: Neural architecture built around attention and feed-forward blocks.

Attention head: One learned attention mechanism within multi-head attention.

Pretraining: Initial large-scale self-supervised model training.

SFT: Supervised fine-tuning on curated examples.

RLHF: Reinforcement learning from human feedback.

DPO: Direct Preference Optimization.

Quantization: Lower-precision representation to reduce memory or compute.

KV cache: Stored attention keys and values for faster autoregressive inference.

Context window: Maximum input-plus-output sequence capacity for a model request.

Tokenizer: Maps text to token IDs and back.

15. Suggested study sequence

- Python, NumPy, linear algebra, probability, and basic calculus.
- Tokenization, embeddings, neural networks, and gradient descent.
- Transformer blocks, attention, positional encodings, and causal masking.
- Pretraining, instruction tuning, preference optimization, and evaluation.
- Prompting, structured outputs, tool calling, and inference controls.
- Run local models with Ollama; compare quality, speed, and memory.
- Build an API, add RAG, evaluate it, and deploy with monitoring.